



CHAPTER 2 · PERFORMANCE CONCEPTS

Computer Organization and Architecture

Designing for Performance

Let's do this efficiently.

Why Performance Still Matters

THE PARADOX

- Cost keeps dropping while performance keeps rising, at the same time.
- A modern laptop has the computing power of a mainframe from a decade or two ago.
- Processors are now cheap enough to throw away after a single use.
- The building blocks are largely the same as 50 years ago. What changed is how cleverly we squeeze performance out of them.

WHAT DEMANDS IT



Image processing and 3D rendering



Videoconferencing and multimedia



Massive client and server networks



Cloud platforms serving many clients at once

Feeding the Processor

Raw transistor speed means nothing if the processor is not kept constantly fed with work. Contemporary processors lean on five techniques:



Pipelining

Overlap instruction stages, like an assembly line, so several instructions are in processing at once.



Branch Prediction

Looks ahead in the instructions and predicts which branches/instructions come next and prefetches to keep the pipeline full.



Superscalar Execution

Issue more than one instruction per clock cycle across multiple parallel pipelines.



Data Flow Analysis

Analyzes and reorder instructions around data dependencies to optimize a schedule instead of using strict program order.

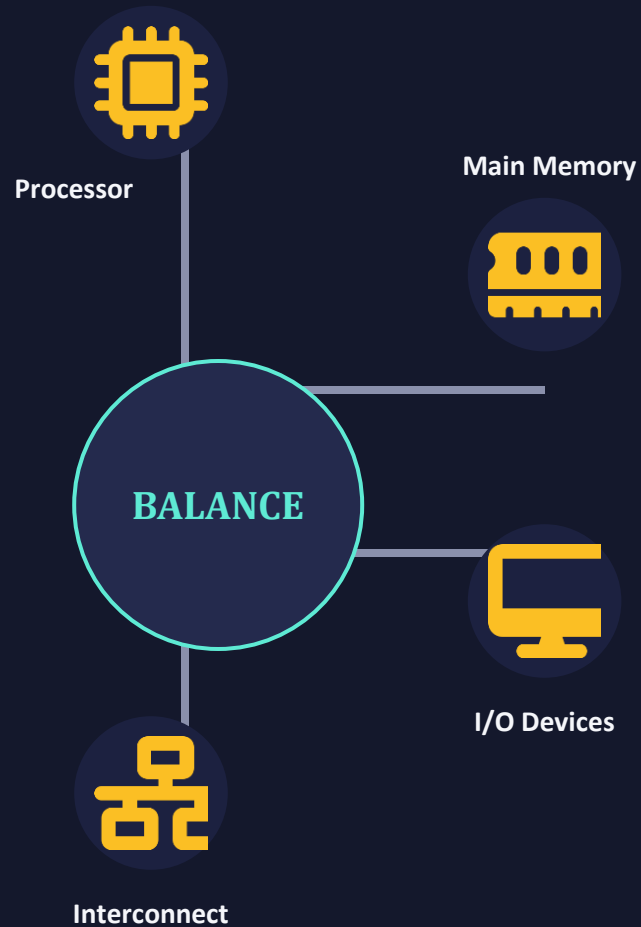


Speculative Execution

Uses branch prediction and data flow analysis to run likely-needed instructions ahead of time and hold the results in reserve.

Performance Balance

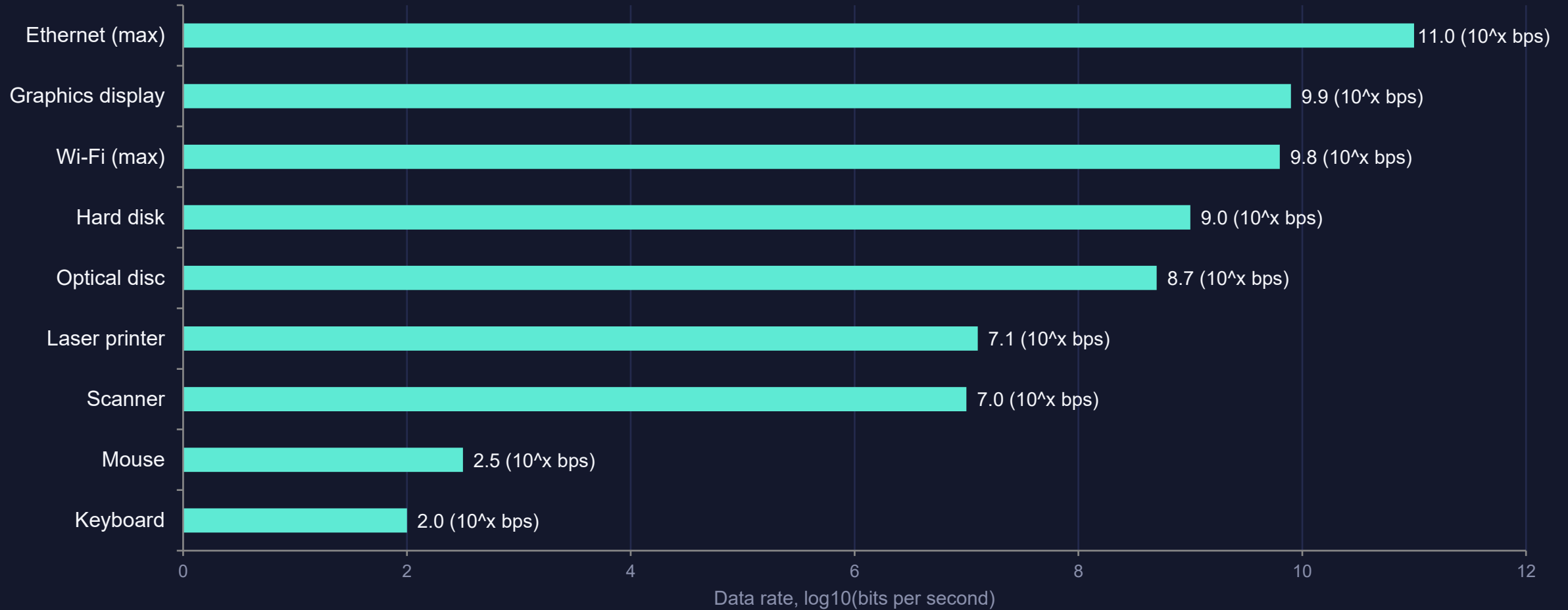
Processor speed raced ahead of memory, I/O, and interconnects (busses). Architects compensate for the mismatch rather than letting the fastest part sit idle waiting on the slowest.



ARCHITECTURAL FIXES

- Widen DRAM and bus data paths to move more bits per access.
- Add on-chip cache and buffering to the DRAM interface itself.
- Layer multiple levels of increasing complex and efficient cache between processor and memory.
- Increase interconnection bandwidth with faster, hierarchical buses to buffer and structure data flow.

I/O Devices Span Ten Orders of Magnitude



Three Ways to Speed Up a Processor



Hardware Speed

Shrink logic gates so more pack together and signals travel less distance, then raise the clock rate.

The clock rate is determined by the slowest signal path. If you shorten the path, you can reconfigure the clock's frequency. However, that is limited because it generates more heat and can cook the chip. (see slide 8)



Bigger, Faster Caches

Dedicate more of the die to cache. On-chip cache access is dramatically faster than off-chip.

Varies, but in general:

L1 – on the core and is private to it

L2 – usually on the core and is private to it

L3 – on the chip but shared among all the cores on that chip; sits between the cores and memory controller



Organization and Architecture

Change how instructions execute, mostly by exploiting parallelism (pipelining and superscalar execution), to raise effective speed without raising the clock.

Branch prediction, data flow analysis, and speculative execution helps feed the parallelism.

Why Clock Speed Hit a Wall



Power

Power density rises with logic density and clock speed. Dissipating that heat becomes a serious design problem.



RC Delay

Signal speed (how fast electrons can flow) is limited by wire resistance (R) and capacitance (C). Shrinking, closely packed wires make this worse, not better.



Memory Latency and Throughput

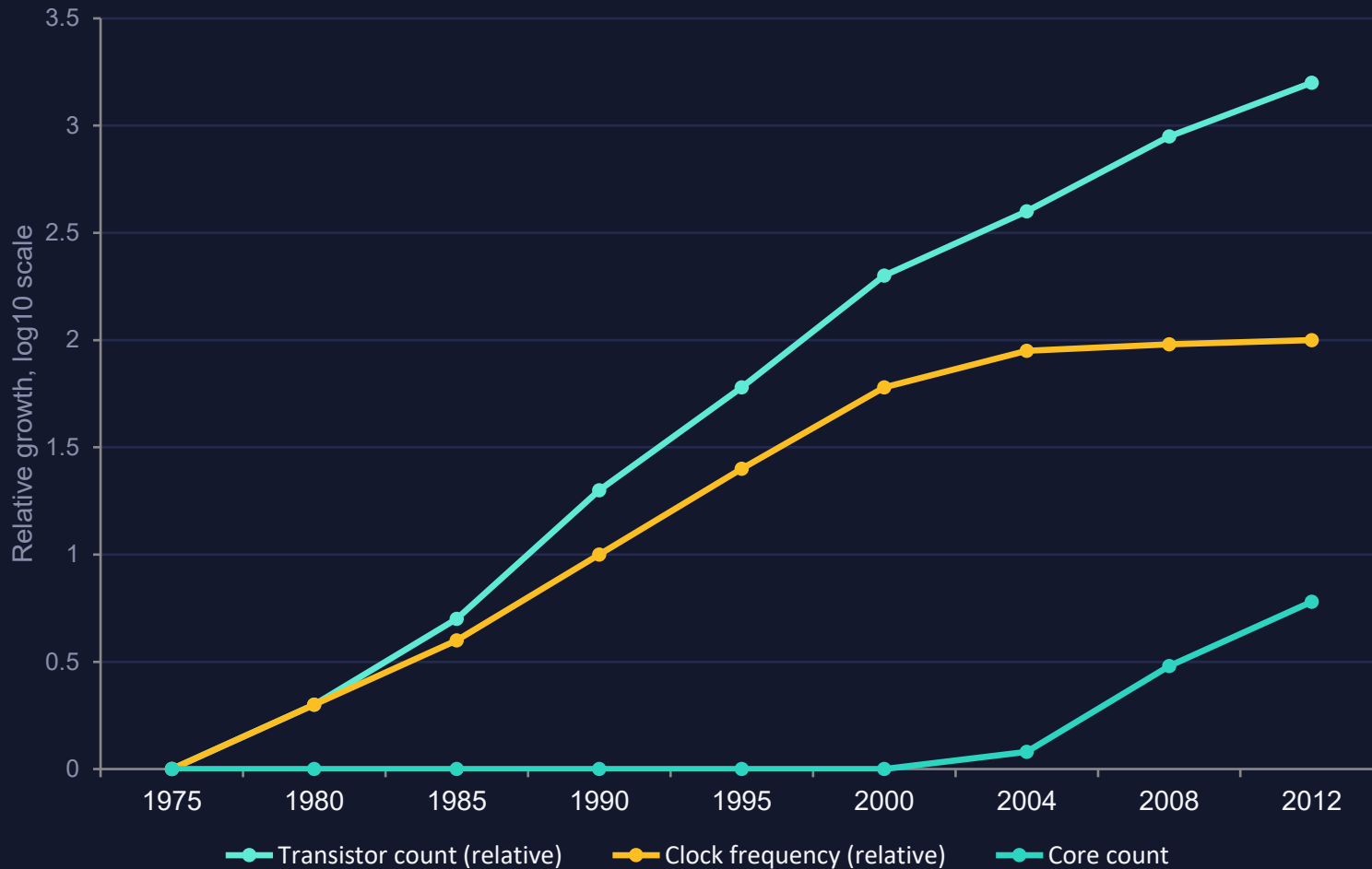
Memory access speed (latency) and transfer speed (throughput) have not kept pace with processor speed, so the processor waits.



SO WHAT?

From the mid-2000s on, most performance gains stop coming from a faster clock. They come from organization and architecture instead, above all, from putting multiple cores on one chip.

Transistors Kept Growing. Clock Speed Did Not.



Frequency plateaus around 2004

Power and heat make it impractical to keep raising the clock rate, even as transistor budgets keep climbing.

Core count rises to compensate

Extra transistors go into more cores instead of a faster single core, the multicore era begins right where frequency flattens.

Multicore: More Simple Cores, Not One Complex Core

Two simpler cores can nearly double performance, **if the software can use both.**



Place multiple processors on one chip sharing a large cache, instead of building one ever more complex core.



With two cores, a larger shared cache is easily justified. Cache logic uses far less power than processing logic.



Two-core chips were quickly followed by four, then eight, then sixteen, as logic density kept rising.

MIC and GPU: More Cores, More Specialization



Many Integrated Core (MIC)

- More than 50 general-purpose cores on a single chip.
- A homogeneous collection of general-purpose processors, not specialized units.
- The leap in raw capability comes with a matching leap in the difficulty of writing software that can actually use that many cores.



Graphics Processing Unit (GPU)

- A core built to run parallel operations on graphics data: encoding, rendering, and video.
- Increasingly used as a vector processor for any workload built from repetitive computation, GPGPU.
- This blurs the line between GPU and CPU, but only for software written to hand off the right work to each.

Hardware Builds the Capacity. Software Has to Use It.



Hardware Potential

Pipelining, superscalar,
multicore, GPU



Software Design

Parallel algorithms,
compiler optimization,
instruction scheduling



Realized Performance

What the user
actually experiences

Every technique in this chapter, pipelining, branch prediction, multicore, GPU, only pays off if software is structured to use it. Amdahl's Law puts an exact number on how much is lost when it is not.

Amdahl's Law

GENE AMDAHL, 1967

- Describes the potential speedup of a program run on multiple processors compared to one processor.
- Shows exactly why building multicore machines is only half the problem: the software must be adapted to a highly parallel environment to exploit that hardware.
- Generalizes to any technical improvement in a system, not just adding processors.
- Speedup = performance (or execution time) for the entire task when enhancements are applied / performance (or execution time) of the same task without those enhancements

SPEEDUP

$$\text{Speedup} = \frac{1}{(1 - f) + f / N}$$

f = fraction of the program that is perfectly parallelizable

1 - f = the serial fraction, which no amount of hardware can speed up

N = number of processors applied to the parallel portion

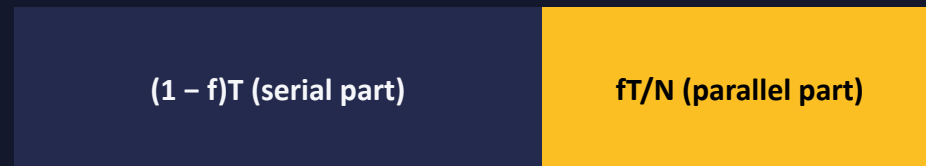
As N grows large, speedup is capped at $1 / (1 - f)$. A tiny serial fraction quietly sets a hard ceiling.

Where the Time Actually Goes

ONE PROCESSOR · TOTAL TIME T

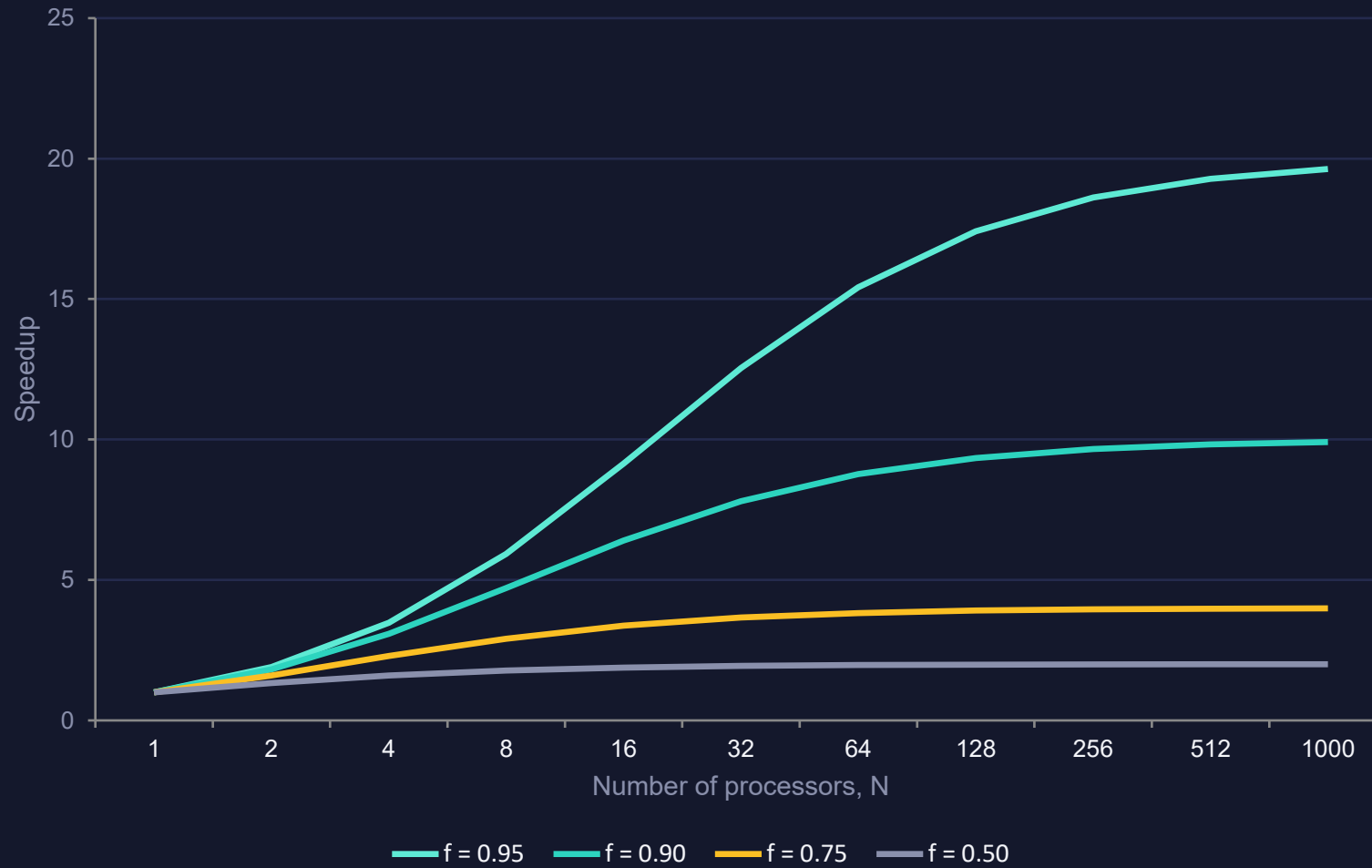


$N = 4$ PROCESSORS · SERIAL PART UNCHANGED, PARALLEL PART DIVIDED BY N



The serial part never shrinks, no matter how many processors you add. Total time collapses toward that fixed slice, which is exactly why the curve on the next slide flattens out.

More Processors, Less and Less Payoff



Even with 95% of a program parallelizable, speedup tops out around 20×.

The remaining 5% that runs serially is what caps the entire curve, no matter how many processors you throw at it.

Four Attributes, Five Factors

System attribute	lc	p	m	k	t
Instruction set architecture	X	X	—	—	—
Compiler technology	X	X	X	—	—
Processor implementation	—	X	—	—	X
Cache and memory hierarchy	—	—	—	X	X

lc = instruction count

p = cycles per instruction

m = memory references per instruction

k = memory-to-cycle time ratio

t = cycle time



Compiler technology touches three of five factors

It is the only system attribute that reaches instruction count, cycles per instruction, and memory references all at once. That is compiler technology, which is software, not silicon.

What Makes a Good Benchmark



Portable

Written in a high-level language so it runs unchanged across different machines.



Representative

Matches a real programming domain: systems, numerical, or commercial workloads.



Measurable

Its results can be captured easily and reproducibly.



Widely Distributed

Used broadly enough that comparisons across vendors and systems actually mean something.

SPEC: A Common Yardstick



Benchmark Suite

A collection of programs, defined in a high-level language, that together give a representative test of a computer in a particular application or system-programming area.



SPEC

The System Performance Evaluation Corporation is an industry consortium that defines and maintains the best-known collection of benchmark suites for evaluating computer systems. Its measurements are widely used for comparison and research.

SPEC CPU2017 in Numbers

The industry-standard suite for processor-intensive applications, measuring performance where computation, not I/O, dominates the workload.

20

Integer Benchmarks

23

Floating-Point Benchmarks

11M+

Lines of Code

3

Written in C, C++, Fortran

For nearly every integer benchmark, and most floating-point ones, SPEC provides both a rate variant and a speed variant.

Performance Concepts

Hardware Techniques



Pipelining, branch prediction, superscalar execution, multicore, MIC, GPU.

Measuring the Limits



Performance balance across components, and Amdahl's Law bounding what parallel hardware can deliver.

Benchmarking



Benchmark principles and the SPEC CPU2017 suite as a common yardstick across vendors.

The Throughline



None of the hardware above pays off unless software, algorithms, and compilers are built to exploit it.



Permissions Notice

This material is adapted from copyrighted instructor resources provided with Computer Organization and Architecture: Designing for Performance, 11th Edition (Pearson). It is provided solely for use in teaching this course and assessing student learning. It should not be redistributed, posted publicly, or shared outside this class, consistent with the publisher's terms for instructor materials.